LIGHT-LEVEL CONTROL WITH AN LED AND LDR

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....1
4.0 Concluding Remarks.....4
Recommended Citation.....4

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Write a discrete-time PI controller as a difference equation and implement it in a MATLAB Function block.	☐
2	Close a feedback loop around an LED and a light-dependent resistor (LDR), and compare proportional-only and PI control.	☐
3	Measure how the closed loop rejects changes in ambient light, and explain the limits of that rejection.	☐

2.0 Hardware Set-Up

This practical exercise builds on the hardware set-up described in the practical exercises, LED (PWM) and LDR (Light Dependent Resistor), which detail how an LED is driven from a PWM output and how an LDR is read through a voltage divider on an analogue input. The set-up is not repeated here.

Place the LED so that it shines directly onto the LDR, about 1 cm away, and cover both with a short tube of card or a folded piece of paper. Leave an opening of about 1 cm at the side, which can be partly covered by hand to change the ambient light reaching the LDR. Only the Base + Sense kit is needed.

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. A PI controller written as a difference equation adjusts the brightness of the LED to hold the LDR reading at a setpoint, even when the ambient light changes. The exercise supports the ACE-CORE learning guide, Discrete-Time PID Control.

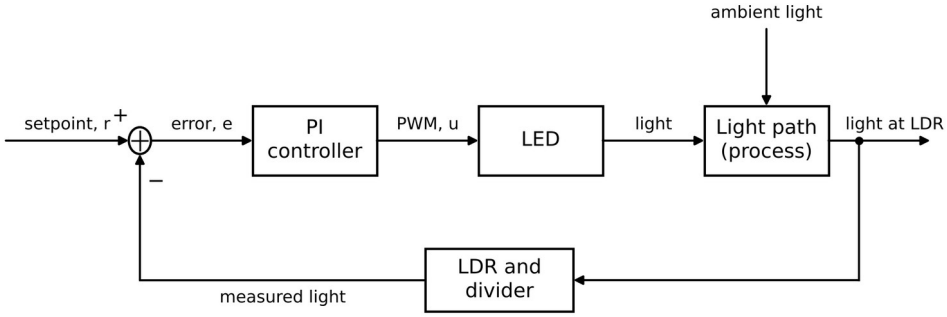


Figure 1: *Closed-Loop Light-Level Control*

Background: PI Control as a Difference Equation

A continuous-time PI controller sets its output from the error $e = r - y$:

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau$$

Differentiating, and approximating the derivatives with backward differences over one sample period T_s, gives the incremental (velocity) form of the discrete-time PI controller:

$$u[k] = u[k-1] + K_p\,(e[k] - e[k-1]) + K_i\, T_s\, e[k]$$

The controller calculates the change in its output at each sample and adds it to the previous output. Limiting the output to the valid PWM range at each sample also prevents the integral action from winding up.

MATLAB Function Block: incremental PI controller with output limits

```
function u = fcn(r, y, Kp, Ki, Ts)
% r: setpoint, y: measured LDR reading (ADC counts), u: PWM value
persistent u_prev e_prev
if isempty(u_prev)
    u_prev = 0;  e_prev = 0;
end
e = r - y;                                          % error
u = u_prev + Kp*(e - e_prev) + Ki*Ts*e;             % incremental PI
u = min(max(u, 0), 255);                            % PWM limits
u_prev = u;  e_prev = e;
```

Find the Operating Range

1. In open loop, step the LED PWM value from 0 to 255 in steps of 51, holding each for 2 s, and log the LDR reading.
2. Record the LDR reading at PWM 0 and PWM 255. Choose a setpoint about halfway between them.
3. Check whether the LDR reading increases or decreases with brightness in your divider. If it decreases, use $e = y - r$ instead of $r - y$ in the function.

Close the Loop

[Screenshot to insert] *Constant (setpoint) and Analog Input (LDR) → MATLAB Function (incremental PI) → PWM output (LED). Scopes log r, y, u and t.*

Figure 2: *Simulink Model for Light-Level Control*

1. Build the model in Figure 2 with T_s = 0.02 s. Start with K_p = 0.2 and K_i = 0 (proportional only).
2. Run the model, apply the setpoint, and record the final LDR reading and the steady-state error in Table 2.
3. Set K_i = 5 and repeat. Record the results.
4. With the PI controller running, partly cover the side opening with your hand for about 5 s, then uncover it. Record how far the reading moves and how long it takes to recover.
5. Increase K_p and K_i step by step and note the gains at which the response starts to oscillate.

Table 2: *Light-Level Control Results*

Controller	K_p	K_i	Final reading	Steady-state error	Recovery after covering (s)
P only	0.2	0			
PI	0.2	5			
PI (higher gains)					

With proportional control alone, the reading settles with an offset from the setpoint, because an error is needed to keep the LED lit. Adding integral action removes the offset. The gains here are starting points: suitable values depend on your LDR, divider and setpoint.

Explore the Limits

1. What happens if the ambient light alone makes the LDR reading pass the setpoint? Explain using the PWM limits.
2. The LDR responds more slowly as the light gets dimmer. Explain why this could make the loop oscillate at high gains.
3. Write the difference equation for a PID controller in the same incremental form.

4.0 Concluding Remarks

This practical exercise implemented a discrete-time PI controller as a difference equation and used it to control the light level measured by an LDR. Using MATLAB, Simulink and the ACE-Lab Base + Sense hardware, proportional-only and PI control were compared, and the controller was shown to compensate for changes in ambient light.

The incremental form of the controller is simple to implement on a microcontroller and limits naturally at the PWM range. The exercise also showed the limits of feedback: a controller cannot reject a disturbance larger than its actuator can correct, and high gains combined with a slow sensor lead to oscillation.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Light-Level Control with an LED and LDR. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://zenodo.org/records/XXXXXXXX